Facultad de Ingeniería y Ciencias
Escuela de Informática y Telecomunicaciones

Sistemas Operativos

# Tarea 2:
## Torneo de Komikan Concurrente

*Integrantes:*
Sebastián Saldivia
Martín Correa

Noviembre de 2026

A continuación se detallan los distintos aspectos de la solución: el esquema de sincronización que protege el tablero de cada mesa, los riesgos de concurrencia identificados y la evaluación de las decisiones de diseño.

# 1. Diseño del Esquema de Sincronización

## 1.1. Arquitectura

El servidor (`server.c`) actúa como *matchmaker* y despachador. Cada partida corre en un **proceso hijo** creado con `fork()`, dentro de él, el hilo principal es el **juez** y se crean 13 hilos de ficha (1 puma y 12 perros). Al ser procesos independientes, las mesas no comparten memoria, por lo que no quedan recursos huérfanos y la caída de una mesa no afecta a las demás.

## 1.2. Emparejamiento y límite de $M$ mesas

Los $N$ jugadores llegan con retardos y roles aleatorios, se encolan según su rol y forman parejas (un puma y un perros) en orden FIFO. El servidor mantiene la tabla `pid_mesas[M]` y solo hace `fork()` si hay una posición libre, el resto espera de forma pasiva hasta que `SIGCHLD` avisa que una mesa terminó

## 1.3. Primitivas y exclusión mutua

Cada mesa tiene un `pthread_mutex_t` por casilla (28), un `lock_juez` que protege el contador de capturas y la posición de cada ficha, y banderas `atomic_int` (`juego_activo`, `fallo_puma`, `fallo_perros`). Cada ficha duerme un tiempo aleatorio en (`min_time`,`max_time`), elige un vecino al azar e intenta moverse. Verificar que el destino está libre y ocuparlo ocurre en una sola sección crítica con los *locks* de origen y destino, por lo que de dos fichas que apuntan a la misma casilla solo una la ocupa. Una captura toma los tres *locks* involucrados (origen, perro y aterrizaje) y revalida las condiciones dentro de la sección crítica. Todo hilo adquiere sus *locks* en **orden ascendente de casilla**, sin duplicados, y los libera en orden inverso. El juez revisa el estado del tablero cada 50ms, imprime el tablero y detecta las victorias (6 capturas, puma sin movimientos o abandono).

## 1.4. Interrupciones

El abandono de un jugador se simula con `SIGUSR1` (puma) o `SIGUSR2` (perros), el manejador solo activa una bandera atómica y el juez declara ganador al oponente, une los hilos y destruye los mutex. Ante Ctrl+C, el manejador de `SIGINT` modifica una variable `volatile sig_atomic_t`, el servidor deja de aceptar jugadores, envía `SIGTERM` a las mesas, las espera con `waitpid`, libera su memoria y termina. Si una mesa muere, `waitpid` lo detecta y libera su posición.

# 2. Análisis de Riesgos de Concurrencia

- **Deadlock:** Dos fichas que intercambian casillas podrían esperarse en círculo, una tiene el lock de la casilla $c_1$ y pide el de $c_2$, mientras que la otra hace lo contrario. Para evitarlo, todos los locks se solicitan en orden ascendente por índice de casilla, y se liberan en orden inverso.
- **Condición de carrera:** El puma puede ver un perro adyacente, pero este puede moverse o ser capturado antes de que el puma tome los locks. Por eso toda decisión se revalida con los locks ya tomados, y las lecturas previas no modifican el estado del tablero.
- **Livelock:** No hay reintentos con `trylock` ni retrocesos, un movimiento fallido simplemente espera al siguiente ciclo, de duración aleatoria, sin conservar ningún lock.
- **Starvation:** Las secciones críticas duran microsegundos y ningún hilo duerme con un lock tomado.

# 3. Evaluación

## 3.1. Granularidad del bloqueo

Un único *mutex* por tablero sería correcto y sin riesgo de *deadlock*, pero afectaría el paralelismo. Un *mutex* por casilla permite avanzar en paralelo a los movimientos sobre casillas disjuntas, a cambio de más complejidad (orden de *locks*, capturas de tres casillas, instantánea con 28 *locks*). Como cada ficha se mueve en promedio cada 175 ms y las secciones críticas duran microsegundos, el *throughput* lo domina el retardo configurado, no la contención.

## 3.2. Proceso por mesa

Aporta aislamiento, permite detectar caídas con `waitpid` y evita memoria compartida.

## 3.3. Juez con sondeo

El juez revisa el estado cada 20ms, un intervalo menor que el retardo mínimo de las fichas, en lugar de usar variables de condición. Esta decisión es más simple de implementar y no agrega sincronización adicional a cada movimiento.

## 3.4. Pruebas y limitaciones

Compila sin advertencias con `gcc -Wall -Wextra -std=c17`. Puesto que el movimiento de las fichas se realiza de forma aleatoria, es muy poco probable que los perros ganen, sin embargo, es posible.